Практическое занятие №4

Авторизация и разграничение прав доступа в СУБД

1. Постановка задачи

Представим, что необходимо защитить базу данных университета.

В базе есть четыре основные таблицы:

```
students
teachers
grades
payments
```

Создадим их.

```
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    full_name VARCHAR(100),
    group_name VARCHAR(50),
    email VARCHAR(100)
);

CREATE TABLE teachers (
    id SERIAL PRIMARY KEY,
    full_name VARCHAR(100),
    department VARCHAR(100)
);

CREATE TABLE grades (
    id SERIAL PRIMARY KEY,
    student_id INT,
    subject VARCHAR(100),
    grade INT
);

CREATE TABLE payments (
    id SERIAL PRIMARY KEY,
    student_id INT,
    amount DECIMAL(10,2),
    payment_date DATE
);
```

Добавим тестовые данные:

```
INSERT INTO students (full_name, group_name, email)
VALUES
('Иванов Иван', 'IT-23', 'ivanov@mail.ru'),
('Петров Петр', 'IT-23', 'petrov@mail.ru'),
('Сидорова Анна', 'IT-24', 'sidorova@mail.ru');
INSERT INTO teachers (full_name, department)
VALUES
('Ким Сергей', 'Информационные технологии'),
('Ахметова Алия', 'Компьютерные науки');
INSERT INTO grades (student_id, subject, grade)
```

```
VALUES
(1, 'Базы данных', 90),
(2, 'Базы данных', 85),
(3, 'Программирование', 95);
```

2. Задание №1. Создание пользователей

Создайте четыре учётные записи:

```
CREATE USER student_user
WITH PASSWORD 'Student_123!';

CREATE USER teacher_user
WITH PASSWORD 'Teacher_123!';

CREATE USER accountant_user
WITH PASSWORD 'Accountant_123!';

CREATE USER db_admin
WITH PASSWORD 'Admin_123!';
```

Назначение пользователей

Пользователь	Назначение
`student_user`	студент
`teacher_user`	преподаватель
`accountant_user`	бухгалтер
`db_admin`	администратор

3. Задание №2. Составление матрицы доступа

Перед выдачей прав студент должен самостоятельно определить, кому что разрешено.

Заполните таблицу:

Роль	students	teachers	grades	payments
Student	?	?	?	?
Teacher	?	?	?	?
Accountant	?	?	?	?
Admin	?	?	?	?

Используйте обозначения:

- **R** — чтение (`SELECT`);
- **C** — создание/добавление (`INSERT`);
- **U** — изменение (`UPDATE`);
- **D** — удаление (`DELETE`);
- **—** — доступа нет.

Рекомендованная модель

После самостоятельной попытки можно сравнить с вариантом:

Роль	students	teachers	grades	payments
Student	R	—	R	—
Teacher	R	R	R/U	—
Accountant	—	—	—	R/U
Admin	R/C/U/D	R/C/U/D	R/C/U/D	R/C/U/D

4. Задание №3. Предоставление прав с помощью GRANT

Теперь реализуем разработанную модель.

Студент

Студенту разрешаем читать студентов и оценки:

```
GRANT SELECT
ON students, grades
TO student_user;
```

Проверяем:

```
SELECT *
FROM students;
```

и:

```
SELECT *
FROM grades;
```

Попробуйте выполнить:

```
UPDATE grades
SET grade = 100
WHERE id = 1;
```

Операция должна быть отклонена.

Почему?

Потому что у `student_user` нет:

```
UPDATE
```

Преподаватель

Предоставим преподавателю необходимые права:

```
GRANT SELECT
ON students, teachers, grades
TO teacher_user;
```

Разрешим изменение оценок:

```
GRANT UPDATE
ON grades
TO teacher_user;
```

Теперь преподаватель может:

```
SELECT *
FROM grades;
```

и:

```
UPDATE grades
SET grade = 95
WHERE id = 1;
```

Но попробуйте:

```
DELETE FROM grades
WHERE id = 1;
```

Операция должна быть запрещена.

Бухгалтер

Бухгалтер работает только с платежами:

```
GRANT SELECT, UPDATE
ON payments
TO accountant_user;
```

Проверьте:

```
SELECT *
FROM payments;
```

Затем попробуйте:

```
UPDATE payments
SET amount = 100000
WHERE id = 1;
```

А после этого:

```
SELECT *
FROM students;
```

Доступ к `students` должен быть запрещён.

5. Задание №4. Отзыв прав с помощью REVOKE

Предположим, преподавателю случайно предоставили право удаления оценок:

```
GRANT DELETE
ON grades
TO teacher_user;
```

Проверьте, что преподаватель теперь может удалить запись.

Затем исправьте ошибку:

```
REVOKE DELETE
ON grades
FROM teacher_user;
```

Снова попробуйте:

```
DELETE FROM grades
WHERE id = 1;
```

Теперь операция должна быть запрещена.

6. Задание №5. Создание ролей

Теперь вместо индивидуальной настройки прав создадим роли.

Создайте:

```
CREATE ROLE student_role;
CREATE ROLE teacher_role;
CREATE ROLE accountant_role;
```

Роль студента

```
GRANT SELECT
ON students, grades
TO student_role;
```

Назначьте роль пользователю:

```
GRANT student_role
TO student_user;
```

Роль преподавателя

```
GRANT SELECT
ON students, teachers, grades
TO teacher_role;
```

```
GRANT UPDATE
ON grades
TO teacher_role;
```

Назначаем роль:

```
GRANT teacher_role
TO teacher_user;
```

Роль бухгалтера

```
GRANT SELECT, UPDATE
ON payments
TO accountant_role;
```

Назначаем:

```
GRANT accountant_role
TO accountant_user;
```

Получается:

```
student_user
      ↓
student_role
      ↓
SELECT students
SELECT grades
```

и:

```
teacher_user
      ↓
teacher_role
      ↓
SELECT students
SELECT teachers
SELECT grades
UPDATE grades
```

7. Задание №6. Проверка принципа минимальных привилегий

Теперь рассмотрите ситуацию:

Преподавателю необходимо только просматривать список студентов и изменять оценки.

Проверьте, нет ли у него лишних прав.

Попробуйте:

```
DELETE FROM students
WHERE id = 1;
DROP TABLE students;
CREATE USER test_user
WITH PASSWORD 'Test_123!';
```

Все эти операции должны быть запрещены.

Сделайте вывод:

`teacher_user` имеет рабочие права, но не имеет административных привилегий.

8. Задание №7. Самостоятельное проектирование

Создайте новую роль:

```
analyst_role
```

Представим, что аналитик должен:

- читать студентов;
- читать оценки;
- читать платежи;
- не изменять данные;
- не удалять данные;
- не создавать пользователей.

Самостоятельно напишите SQL-команды.

Ожидаемый вариант:

```
CREATE ROLE analyst_role;

GRANT SELECT
ON students, grades, payments
TO analyst_role;
```

Создайте пользователя:

```
CREATE USER analyst_user
WITH PASSWORD 'Analyst_123!';
```

Назначьте роль:

```
GRANT analyst_role
TO analyst_user;
```

Проверьте:

```
SELECT * FROM students;
```

и попробуйте:

```
UPDATE students
SET group_name = 'IT-25'
WHERE id = 1;
```

Вторая операция должна быть запрещена.

9. Итоговая проверка

должны проверить следующие операции.

Пользователь	SELECT	INSERT	UPDATE	DELETE
Student	✓	✗	✗	✗
Teacher	✓	✗	✓	✗
Accountant	✓*	✗	✓*	✗
Analyst	✓	✗	✗	✗
Admin	✓	✓	✓	✓